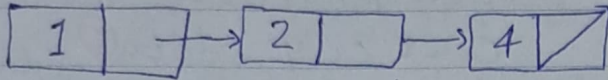


شماره دانشجویی: ۴۰۱۱۸۳۳۲۳۰

کلی فاضل ضیا

تمرین ۱-۲: الگوریتمی بنویسید که عدد x را طوری به لیست پیوندی مرتب شده اضافه کند که ترتیب عناصر لیست بهم نریزد. الگوریتمی ارائه شده را برای لیست زیر و بازایی $x=3$ ، مرحله به مرحله اجرا کنید (trace)

Start ↓



void insert (node **start, int n)

{

node *p, *q, *t = new node();

t->data = n

if (*start == Null || n <= (*start)->data)

{

t->next = *start;

*start = t;

}

*p = *start;

q = p->next;

while (q && q->data < n)

{ p = q;

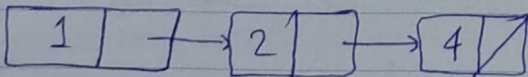
q = q->next;

p->next = t;

t->next = q;

الگوریتم ارائه شده را برای لیست زیر trace کنید.

Start ↓



~~void insert (node **start, int n)~~

int back_print (node *start)

{

if (start)

{ back_print (start->next);

cout << start->data

Parsian }

return 0;

}

صفحه اول

صفحه دوم

trace که برای $x=3$ در

لیست پیوندی بالا در آخر صفحه

چهارم

اگر لیست خالی باشد یا

n از همه عناصر کوچکتر

باشد، n را به در جایگاه

node استارت می گذاریم.

به گویی به اول لیست اضافه

می کنیم.

در گام این صورت، از اولین نود شروع

می کنیم و در هر مرحله حلقه بهی مقدار

نود بعد را با n مقایسه می کنیم. اگر مقدار بیشتر

بود به پیمایش ادامه می دهیم تا زمانی که n

از مقدار نود بعدی کمتر باشد. در ادامه نود دارای مقدار n را در جایگاه نود بعد از p

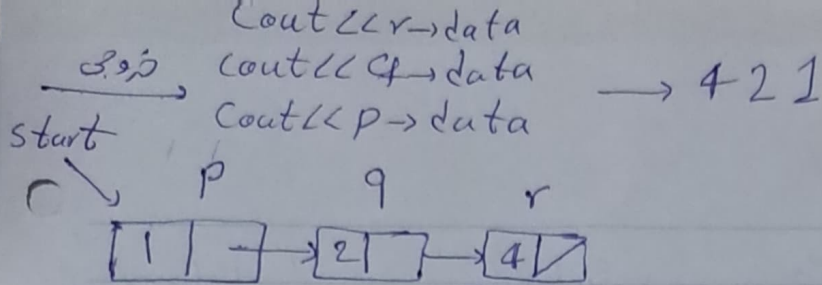
جایگذاری می کنیم.

تمرین ۲-۲: الگوریتم بازگشتی بنویسید که عناصر لیست پیوندی را از آخر به اول چاپ کند.

الگوریتم ارائه شده را برای لیست زیر trace کنید.

الگوریتم:

تیم برای لیست پیوندی در صفحه بعد



$\text{start} = p$ $\text{if}(\text{start}) = \text{True}$ $\text{back_print}(\text{start} \rightarrow \text{next})$
 $\text{back_print}(\text{start})$ $\text{cout} \ll p \rightarrow \text{data}$

$\text{back_print}(p \rightarrow \text{next})$ $\text{if}(q) = \text{True}$ $\text{back_print}(\text{start} \rightarrow \text{next})$
 $\text{cout} \ll q \rightarrow \text{data}$

$\text{back_print}(q \rightarrow \text{next})$ $\text{if}(r) = \text{True}$ $\text{back_print}(r \rightarrow \text{next})$
 $\text{cout} \ll r \rightarrow \text{data}$

$\text{back_print}(r \rightarrow \text{next})$ $\text{if}(\text{null}) = \text{false}$

تمرین ۲-۳: الگوریتم محاسبه بازگشتی پیوندی را از آخر به اول چاپ کند.
 الگوریتم وارانه شده را برای لیست بالایی trace کنید.

void back_print(node *start)

{

node *p = start, *finall = NULL;

while(start != finall)

{

while(p->next != finall)

p = p->next;

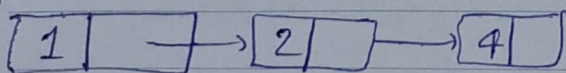
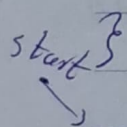
cout << p->data;

finall = p;

p = start

}

Trace برای لیست زیر:



$p \rightarrow \text{next} = [2]$

دور اول: $p = [1]$ $\text{start} \neq \text{finall}$ $p \rightarrow \text{next} \neq \text{finall} \rightarrow p = [2]$

$\text{finall} = \text{Null}$ $p \rightarrow \text{next} \neq \text{finall} \rightarrow p = [4]$ $p \rightarrow \text{next} = \text{finall} \times \rightarrow$

$p \rightarrow \text{next} = [4]$ $\text{cout } 4$ $\text{finall} = [4]$ $p = [1]$

دور دوم: $p = [1]$ $\text{start} \neq \text{finall}$ $p \rightarrow \text{next} \neq \text{finall} \rightarrow p = [2]$

$\text{finall} = [4]$ $p \rightarrow \text{next} = \text{finall}$ $p \rightarrow \text{next} = [2]$

$\rightarrow p \rightarrow \text{next} = \text{finall}$ $\times \rightarrow \text{cout } 2 \rightarrow \text{finall} = [2], p = [2]$

Parsian

از به درصفت نویسه

صفحه سوم

$p \rightarrow next = [2]$
 دور سوم $p = [1]$ ، $start = final$ ، $p \rightarrow next = final \times$
 $final = [2]$ $\rightarrow$ $count = 1$ ، $final = [2]$ ، $p = [1]$

دور چهارم $p = [2]$ ، $start \neq final \times \rightarrow$ به تا به پايان می رسد
 $final = [2]$
 $count = 4$ ، $count < 2$ ، $count = 421$ خروجی:

تمرین ۲۴: الگوریتمی بنویسید که عدد n را به ابتدای لیست پیوندی اضافه کند.

int insert($node^{**}start$, $int n$)
 {

$node^{*}p$, $node^{*}t = new\ node();$

$t \rightarrow data = n;$

$if (start == Null)$

{

$*start = t;$

$(*start) \rightarrow next = *start;$

}

$p = *start;$

$while (p \rightarrow next != *start)$

$p = p \rightarrow next;$

$p \rightarrow next = t;$

$t \rightarrow next = *start;$

$return\ 0;$

}

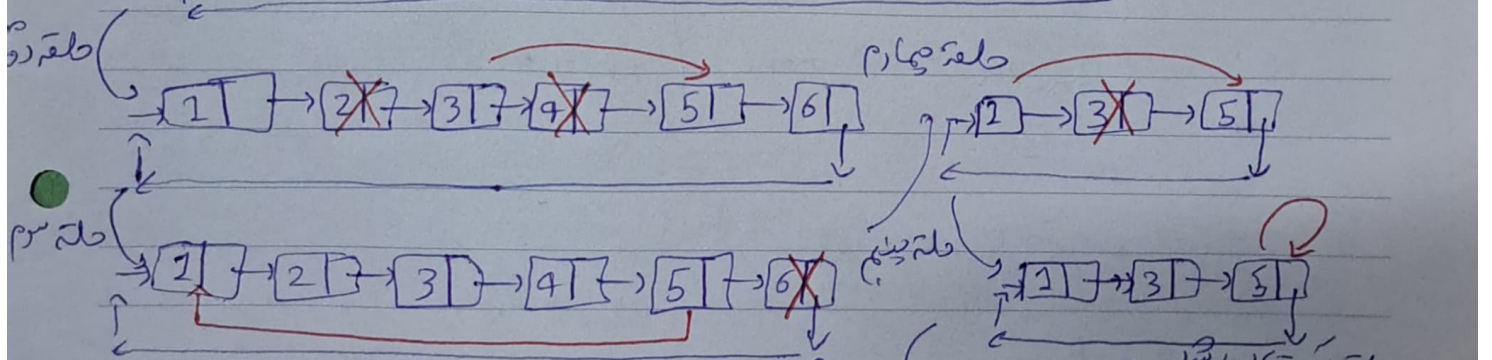
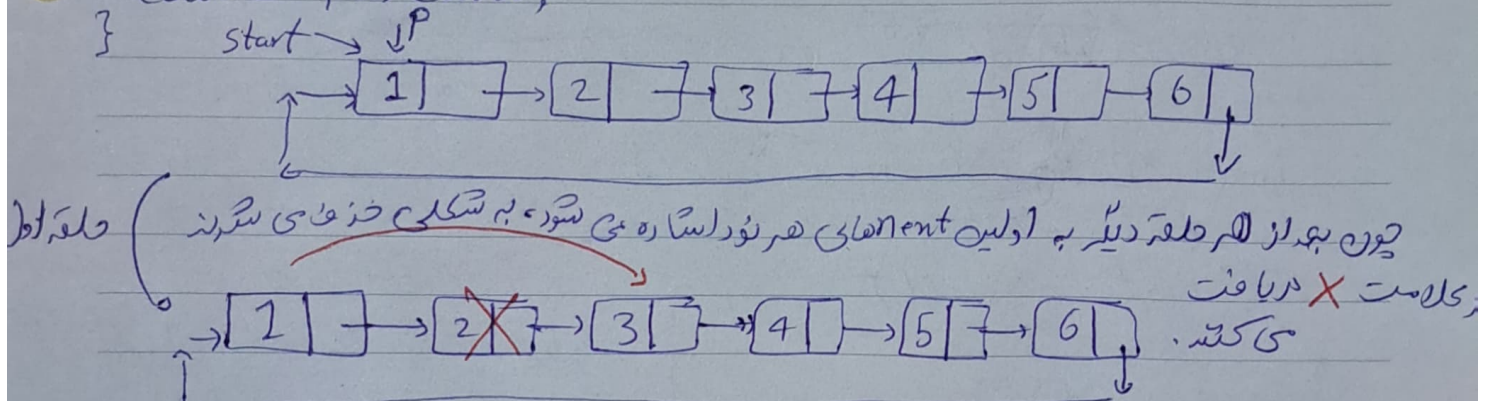
اگر لیست خالی باشد، $node\ t$ را به
 عنوان استارت قرار می دهیم و مقدارش را
 همان n می گذاریم و $start$ آن را به
 خودش می پرد (به این دلیل که لیست پیوندی است)

اگر لیست خالی نباشد، p شرط $while$
 که داریم، با $node\ p$ تا می node خالی
 لیست را به پیمایش می کنیم تا به آخرین $node$
 برسیم، از آنجایی که $next$ آخرین $node$
 به $start$ اضافه می کند، به جای $start$

$node\ t$ را قرار می دهیم و $t \rightarrow next$ به $start$ می پرد. به این صورت $node\ t$ جای
 $start$ را می گیرد. به عبارتی $start$ را به جایی که $start$ قرار می گیرد.

تمرین ۲-۵: ضروی الگوریتم زیر را روی لیست پیوندی مطلق داده شده پیرا کنید.

```
void f(node *start)
{
    node *p = start;
    while (p->next != p)
    {
        p->next = p->next->next;
        p = p->next;
    }
    cout << p->data;
}
```



حالت پنجم: لیست پیوندی مطلق داده شده

```

graph LR
    start --> p
    p --> 1
    1 --> 2
    2 --> 3
    3 --> 4
    4 --> 5
    5 --> 6
    6 --> null
  
```

حالت ششم: لیست پیوندی مطلق داده شده

```

graph LR
    start --> p
    p --> 1
    1 --> 2
    2 --> 3
    3 --> 4
    4 --> 5
    5 --> 6
    6 --> null
  
```

حالت هفتم: لیست پیوندی مطلق داده شده

```

graph LR
    start --> p
    p --> 1
    1 --> 2
    2 --> 3
    3 --> 4
    4 --> 5
    5 --> 6
    6 --> null
  
```

